EXPERMENT: Map Coloring using Constraint Satisfaction Problem (CSP)

Aim

To implement the **Map Coloring problem using a Constraint Satisfaction Problem (CSP)** in Python, where adjacent regions are assigned different colors.

Objectives

1. To understand the concept of **Constraint Satisfaction Problems**.
2. To represent regions and their relationships using a graph.
3. To assign colors while satisfying all constraints.
4. To implement **backtracking** for finding a valid solution.
5. To understand how CSP can solve real-world problems.

Algorithm

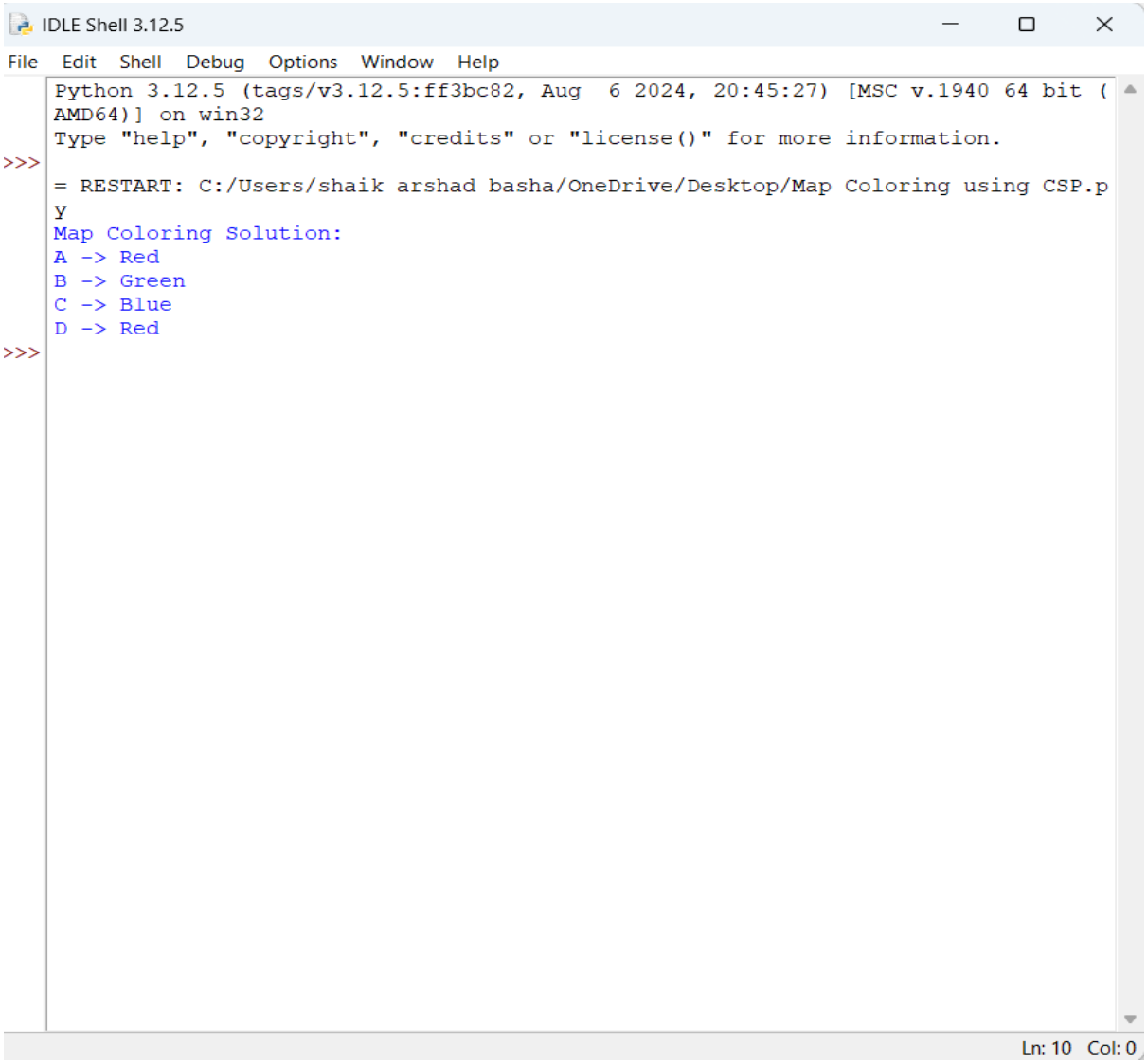
1. Define the regions of the map as variables.
2. Define a set of available colors as the domain.
3. Represent neighboring regions using a graph.
4. Select an unassigned region.
5. Assign a color to the selected region.
6. Check whether the color conflicts with any neighboring region.
7. If there is no conflict, continue with the next region.
8. If a conflict occurs, remove the color and try another color.
9. Use backtracking when no valid color is available.
10. Stop when all regions are successfully colored.

CODE

```
# Map Coloring using Constraint Satisfaction Problem (CSP)

colors = ['Red', 'Green', 'Blue']

# Map represented as a graph

graph = {

   'A': ['B', 'C'],

   'B': ['A', 'C', 'D'],

   'C': ['A', 'B', 'D'],

   'D': ['B', 'C']

}
```

```
assignment = {}
def is_valid(region, color):
    for neighbor in graph[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True
def solve():
    if len(assignment) == len(graph):
        return True
    # Select an unassigned region
    region = next(r for r in graph if r not in assignment)
    for color in colors:
        if is_valid(region, color):
            assignment[region] = color
            if solve():
                return True
            # Backtracking
            del assignment[region]
    return False
if solve():
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(region, "->", color)
else:
    print("No solution exists.")
```

OUTPUT



Result

The Map Coloring problem was successfully implemented using **CSP and backtracking**, and a valid color assignment was obtained without assigning the same color to adjacent regions.

Conclusion

The experiment demonstrates that **Constraint Satisfaction Problems** can efficiently solve map-coloring problems by combining variables, domains, constraints, and backtracking.